

Business Research Agent

AI Solutions Architecture Case Study #1: I identified a business problem, evaluated an AI-based solution, designed the architecture, implemented a constrained MVP, established verification controls, tested it, analyzed its limitations, and documented the architectural trade-offs.

Role: David El Corderu Bey, PhD, Solutions Architect & Project Owner **Implementation partner:** Claude (Senior AI Engineer / pair programmer) **Program:** 12-Week AI Solutions Architect Mastery — a self-directed curriculum built by doing real architecture work and documenting it as it happens, not by following a tutorial.

The Business Problem

Small medical practices — chiropractic offices as the initial use case — lose hours manually researching their market, competitors, and reputation instead of running their business.

The Solution

A single AI agent that takes a plain-English business question, gathers real evidence from the live web, and returns a structured, source-verified research brief: executive summary, key findings, opportunities, risks, and sources.

Architectural Approach: Justify Every Component

The governing constraint for this project, set before a line of code was written: **one agent, one research tool, no database, no RAG, no multi-agent orchestration, no unnecessary framework — and every departure from that has to be justified, not assumed.**

That's not a limitation born of inexperience. It's the actual architectural discipline being practiced this week: a database is the right answer when you need to persist and query state across runs — this system doesn't yet. RAG is the right answer when you're searching a fixed private corpus — this system searches the live web instead, so there's nothing to index. Multiple agents earn their complexity when a task genuinely needs independent specialists reasoning in parallel — one agent, given a clear question and the right tools, already satisfies every success criterion below. Adding any of these without that justification would be complexity sold to a problem that

never asked for it, and every one of them would be one more thing to explain, secure, and maintain.

How It Works

Question -> Agent reads it, decides a search query -> Real web search tool runs, returns actual URLs + content -> Agent reasons over the evidence (may search again if thin) -> Agent submits a structured brief through a second tool call -> Code verifies every cited source against what was actually found -> Verified report saved to file + summary pushed to Telegram

The second tool — a structured "submit your answer" call, rather than free text — is the detail that makes the next section possible at all.

The Decision That Actually Mattered: Verification, Not Trust

The first draft of this architecture asked the model, in its instructions, to only cite real sources. That's a request. A model can misremember, or fill a gap with something plausible-sounding, and a prompt instruction alone can't stop that — it can only ask nicely.

The fix wasn't a better prompt. It was moving the answer out of prose and into structured data, then writing a deterministic check: every URL the model claims is compared, in plain code, against the exact set of URLs the search tool actually returned during that run. No fuzzy matching, nothing to interpret — a URL is either in that set or it isn't. Below the required minimum, the report ships with a visible warning instead of a quiet false pass.

This is the difference between a system that *asks* an LLM to be reliable and one that is reliable regardless of what the LLM does — and it's the single change that turned this from a demo into something a business could actually trust.

Proof, Not a Claim

Live run, real Claude API call, real web search, no mocks:

Question: *"What local marketing tactics work best for a new chiropractic office trying to attract its first patients?"* **Result:** 13 sources cited, **13 verified** against real search results. Exit code 0.

Findings included specific, checkable tactics (Google Business Profile optimization, local referral partnerships, first-visit offers) each tied to a real, clickable source — not summarized folklore.

What Was Deliberately Not Built

Not built	Why
Database	Nothing needs to persist or be queried across runs yet
RAG / vector store	Live web search, not a fixed corpus — nothing to index
Multi-agent orchestration	One agent's own tool-use decisions already meet every success criterion
Framework (LangChain, etc.)	Direct API calls keep the actual mechanics visible — the whole point of Week 1

Known Open Item

The "business research only, never clinical advice" rule currently lives in the prompt only — it's requested of the model, not enforced in code, unlike the source-verification rule. That asymmetry is tracked deliberately, not hidden: it's the next piece of code to write if this boundary needs to be as bulletproof as source verification is.

Skills Demonstrated

- Designing an LLM tool-use loop from the API's raw mechanics, not a framework abstraction
- Turning free-text model output into verifiable structured data via a second tool call
- Building a deterministic guardrail against LLM hallucination, rather than trusting a prompt
- Scoping a system to the smallest architecture that satisfies real requirements, with each addition justified against a stated constraint
- Working inside an approval-gated architecture process: propose -> get explicit sign-off -> implement -> verify live -> report back